

INTRODUCCION A LA PROGRAMACION
TAREA BASES DE DATOS
Alumno: Cristian Padilla Sanchez

Proyecto: Sistema de Gestión de Reparaciones Tecnológicas

Objetivo: Diseñar e implementar una base de datos relacional funcional utilizando SQL, analizando la jerarquía de datos y garantizando la persistencia mediante procedimientos almacenados.

Fase 1: Investigación y Análisis del Diseño

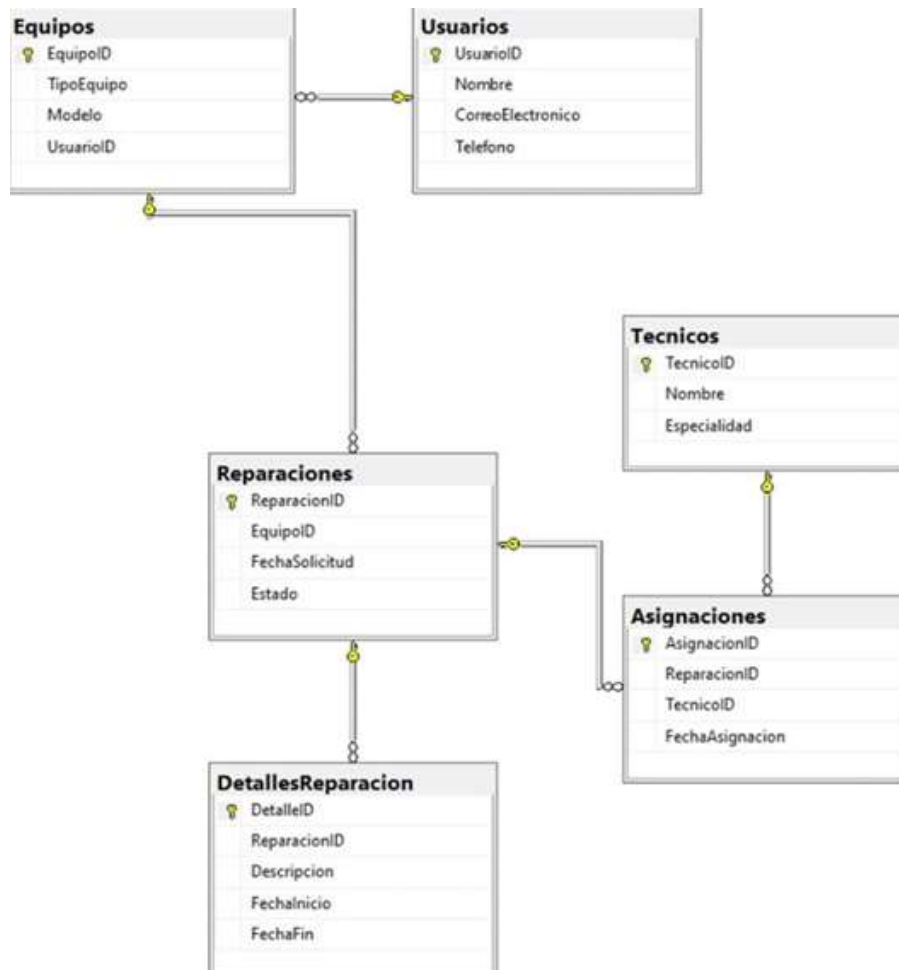
Antes de escribir código, los grupos deben analizar el diagrama proporcionado en el archivo Proyecto programacion - HTML-CSS-BD.pdf.

Mapeo de Atributos: Identifiquen cada campo (columna) y decidan el tipo de dato adecuado (ej. int, varchar(x), datetime) basándose en las mejores prácticas de estandarización.

Integridad Referencial: Identifiquen las Llaves Primarias (PK) y las Llaves Foráneas (FK) que vinculan las 6 tablas (Equipos, Usuarios, Tecnicos, Reparaciones, Asignaciones, DetallesReparacion).

Pregunta de Análisis: Expliquen por qué es más eficiente este modelo relacional que tener un único "archivador electrónico" con toda la información duplicada, citando el concepto de redundancia de datos.

DIAGRAMA DE LAS TABLAS DE LA BASE DE DATOS



1. Mapeo de atributos de las tablas

Tabla Usuarios

Campo	Tipo de dato recomendado	Descripción / Restricción
UsuarioID	INT	Llave Primaria (PK), identificador único del usuario, como el caso del número de cédula
Nombre	VARCHAR(100)	Nombre completo del usuario
CorreoElectronico	VARCHAR(150)	Correo, debe ser único
Telefono	VARCHAR(20)	Número de contacto (permite caracteres especiales)

Tabla Equipos

Campo	Tipo de dato recomendado	Descripción / Restricción
EquipoID	INT	PK, identificador único del equipo
TipoEquipo	VARCHAR(50)	Ej: Una Laptop, impresora, celular, etc.
Modelo	VARCHAR(100)	Modelo del equipo
UsuarioID	INT	Llave Foránea (FK) hace referencia a Usuarios(UsuarioID)

Tabla Reparaciones

Campo	Tipo de dato recomendado	Descripción / Restricción
ReparacionID	INT	PK, identificador único de la solicitud
EquipoID	INT	FK Llave foránea hace referencia a Equipos(EquipoID)
FechaSolicitud	DATETIME	Fecha y hora en que se registra la reparación
Estado	VARCHAR(30)	Ej: Pendiente, En proceso, Finalizada, Cancelada

Tabla Detalles Reparacion

Campo	Tipo de dato recomendado	Descripción / Restricción
DetalleID	INT	PK, identificador único del detalle
ReparacionID	INT	FK hace referencia a Reparaciones(ReparacionID)
Descripcion	TEXT	Descripción detallada del fallo o trabajo realizado
FechaInicio	DATETIME	Fecha en que se inicia la reparación
FechaFin	DATETIME	Fecha en que se finaliza la reparación

Tabla Tecnicos

Campo	Tipo de dato recomendado	Descripción / Restricción
TecnicoID	INT	PK, identificador único del técnico
Nombre	VARCHAR(100)	Nombre completo del técnico
Especialidad	VARCHAR(80)	Área de conocimiento: software, hardware, redes, etc.

Tabla Asignaciones

Campo	Tipo de dato recomendado	Descripción / Restricción
AsignacionID	INT	PK, identificador único de la asignación
ReparacionID	INT	FK hace referencia a Reparaciones(ReparacionID)
TecnicoID	INT	FK hace referencia a Tecnicos(TecnicoID)
FechaAsignacion	DATETIME	Fecha en que se asigna el técnico a la reparación

2. Integridad Referencial: Relaciones entre Tablas

El modelo cumple con las reglas de integridad para garantizar que los datos sean coherentes:

- **Usuarios ↔ Equipos: Relación 1 a N**
 - Un usuario puede registrar varios equipos; cada equipo pertenece a un solo usuario.
 - FK: Equipos.UsuarioID → Usuarios.UsuarioID
- **Equipos ↔ Reparaciones: Relación 1 a N**
 - Un equipo puede tener muchas reparaciones; cada reparación corresponde a un solo equipo.
 - FK: Reparaciones.EquipoID → Equipos.EquipoID
- **Reparaciones ↔ DetallesReparacion: Relación 1 a N**
 - Una reparación puede tener varios registros de detalle (ej: pasos, cambios de piezas); cada detalle pertenece a una sola reparación.
 - FK: DetallesReparacion.ReparacionID → Reparaciones.ReparacionID
- **Reparaciones ↔ Asignaciones: Relación 1 a N**
 - Una reparación puede ser asignada a uno o varios técnicos; cada asignación corresponde a una sola reparación.
 - FK: Asignaciones.ReparacionID → Reparaciones.ReparacionID
- **Técnicos ↔ Asignaciones: Relación 1 a N**
 - Un técnico puede tener muchas asignaciones; cada asignación corresponde a un solo técnico.
 - FK: Asignaciones.TecnicoID → Tecnicos.TecnicoID

3. . **Pregunta de Análisis: Expliquen por qué es más eficiente este modelo relacional que tener un único "archivador electrónico" con toda la información duplicada, citando el concepto de redundancia de datos.**

La redundancia ocurre cuando la misma información se repite en múltiples registros, sin necesidad.

Si todo estuviera en una sola tabla, cada vez que registres una reparación tendrías que escribir nuevamente el nombre, correo y teléfono del usuario; el tipo y modelo del equipo; los datos del técnico, etc. Y esto produce:

- **Desperdicio de espacio:** Se almacena la misma información cientos o miles de veces.
- **Riesgo de inconsistencias:** Si se cambia el correo de un usuario, tendría que modificarlo en todas las filas donde aparezca; si se olvida alguna, los datos quedan desactualizados.
- **Consultas más lentas:** Al tener registros muy largos y repetitivos, las búsquedas, filtros y ordenamientos requieren más tiempo y recursos.
- **Mayor probabilidad de errores:** Al escribir manualmente los mismos datos, aumentan los errores de escritura.

Algunas de las ventajas de utilizar de un modelo relacional, proporciona las siguientes ventajas vs. Un solo archivador.

- **Elimina la redundancia:** Cada dato se guarda una sola vez en su tabla correspondiente, y se vincula mediante claves.
- **Integridad de datos:** Las claves foráneas aseguran que no se pueda registrar un equipo sin usuario, ni una reparación sin equipo, etc.
- **Facilidad de actualización:** Si se cambia el teléfono de un usuario, solo se modifica un registro en Usuarios, y el cambio se refleja automáticamente en todas las reparaciones asociadas.
- **Mayor eficiencia:** Las consultas son más rápidas, ocupan menos espacio y se adaptan mejor al crecimiento de los datos.

En resumen, utilizar un modelo relacional de bases de datos simplifica, ofrece mayor accesibilidad, mayor flexibilidad, organiza de mejor manera, limita errores a la hora de registrar la información.

Fase 2: Creación de la Estructura (DDL)

Fase 2: Creación de la Estructura (DDL)

Utilizando lenguaje de definición de datos (DDL), los estudiantes deben redactar el script para:

Crear la Base de Datos: CREATE DATABASE SistemaReparaciones;

Crear las Tablas: Implementar las sentencias CREATE TABLE respetando las restricciones de integridad (NOT NULL, UNIQUE, IDENTITY).

Ejemplo técnico: La tabla Usuarios debe asegurar que el CorreoElectronico sea único para evitar registros duplicados.

```
-- =====
-- FASE 2: CREACIÓN DE LA ESTRUCTURA (DDL)
-- SISTEMA DE GESTIÓN DE REPARACIONES TECNOLÓGICAS
-- =====

/* Comandos mas utilizados */
--DDL Data definition language: Create, Drop, Truncate, Alter

-- Crear la base de datos
CREATE DATABASE SistemaReparaciones -- Crea el espacio de almacenamiento para el sistema
GO

-- Seleccionar la base de datos para trabajar
USE SistemaReparaciones -- Indica que todas las siguientes acciones se harán en esta base
GO

-- =====
-- Tabla: Usuarios
-- =====
CREATE TABLE Usuarios
(
    UsuarioID    INT PRIMARY KEY IDENTITY(1,1), -- Identificador único autoincremental
    Nombre       VARCHAR(100) NOT NULL,        -- Campo obligatorio
    CorreoElectronico VARCHAR(150) NOT NULL UNIQUE, -- Obligatorio y único para evitar duplicados
    Telefono     VARCHAR(20) NULL              -- Campo opcional
)
GO

-- =====
-- Tabla: Equipos
-- =====
CREATE TABLE Equipos
(
    EquipoID     INT PRIMARY KEY IDENTITY(1,1), -- Identificador único autoincremental
    TipoEquipo   VARCHAR(50) NOT NULL,         -- Tipo de equipo (ej: Laptop, Impresora) - obligatorio
    Modelo       VARCHAR(100) NOT NULL,        -- Marca y modelo del equipo - obligatorio
    UsuarioID    INT NOT NULL,                 -- Enlace al dueño del equipo - obligatorio
    FOREIGN KEY (UsuarioID) REFERENCES Usuarios(UsuarioID) -- Relación con la tabla Usuarios
)
GO
```

```

-- =====
-- Tabla: Tecnicos
-- =====
CREATE TABLE Tecnicos
(
    TecnicoID    INT PRIMARY KEY IDENTITY(1,1), -- Identificador único autoincremental
    Nombre       VARCHAR(100) NOT NULL,        -- Nombre completo del técnico - obligatorio
    Especialidad  VARCHAR(80) NOT NULL         -- Área de conocimiento - obligatorio
)
GO

-- =====
-- Tabla: Reparaciones
-- =====
CREATE TABLE Reparaciones
(
    ReparacionID INT PRIMARY KEY IDENTITY(1,1), -- Identificador único autoincremental
    EquipoID     INT NOT NULL,                 -- Enlace al equipo a reparar - obligatorio
    FechaSolicitud DATETIME NOT NULL,          -- Fecha de solicitud, se ingresa manualmente
    Estado       VARCHAR(30) NOT NULL,         -- Estado de la reparación, se ingresa manualmente
    FOREIGN KEY (EquipoID) REFERENCES Equipos(EquipoID) -- Relación con la tabla Equipos
)
GO

-- =====
-- Tabla: Asignaciones
-- =====
CREATE TABLE Asignaciones
(
    AsignacionID INT PRIMARY KEY IDENTITY(1,1), -- Identificador único autoincremental
    ReparacionID INT NOT NULL,                 -- Enlace a la reparación - obligatorio
    TecnicoID    INT NOT NULL,                 -- Enlace al técnico encargado - obligatorio
    FechaAsignacion DATETIME NOT NULL,         -- Fecha de asignación, se ingresa manualmente
    FOREIGN KEY (ReparacionID) REFERENCES Reparaciones(ReparacionID), -- Relación con Reparaciones
    FOREIGN KEY (TecnicoID) REFERENCES Tecnicos(TecnicoID) -- Relación con Tecnicos
)
GO

-- =====
-- Tabla: DetallesReparacion
-- =====
CREATE TABLE DetallesReparacion
(
    DetalleID    INT PRIMARY KEY IDENTITY(1,1), -- Identificador único autoincremental
    ReparacionID INT NOT NULL,                 -- Enlace a la reparación - obligatorio
    Descripcion   VARCHAR(500) NOT NULL,       -- Detalle de la falla o trabajo - obligatorio
    FechaInicio   DATETIME NULL,               -- Fecha de inicio, se ingresa manualmente
    FechaFin      DATETIME NULL,               -- Fecha de finalización, se ingresa manualmente
    FOREIGN KEY (ReparacionID) REFERENCES Reparaciones(ReparacionID) -- Relación con Reparaciones
)
GO

```

Fase 3: Manipulación y Lógica de Negocio (DML y Procedimientos)

El software debe permitir realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar).

Poblamiento: Insertar al menos 5 registros de prueba en cada tabla usando INSERT INTO.

En "color amarillo" lo que no se va a realizar, indicado por el profesor.

-- =====

-- FASE 3: MANIPULACIÓN Y LÓGICA DE NEGOCIO

-- SISTEMA DE GESTIÓN DE REPARACIONES TECNOLÓGICAS

-- =====

-- =====

-- 1. POBLAMIENTO DE TABLAS

-- Insertar al menos 5 registros en cada tabla

-- =====

-- Insertar en Usuarios

INSERT INTO Usuarios (Nombre, CorreoElectronico, Telefono)

VALUES

('Carlos Ruiz', 'carlos.ruiz@correo.com', '8765-1122'),

('María Gómez', 'maria.gomez@correo.com', '8901-3344'),

('Luis Fernández', 'luis.fernandez@correo.com', '8654-5566'),

('Ana Castro', 'ana.castro@correo.com', '8876-7788'),

('Jorge Mora', 'jorge.mora@correo.com', '8543-9900')

GO

-- Insertar en Equipos

INSERT INTO Equipos (TipoEquipo, Modelo, UsuarioID)

VALUES

('Laptop', 'Lenovo IdeaPad 3', 1),

('Impresora', 'Epson L3250', 2),

('Celular', 'iPhone 13', 3),

('Tablet', 'Samsung Tab S6', 4),

('PC Escritorio', 'HP Pavilion', 5)

GO

-- Insertar en Técnicos

INSERT INTO Tecnicos (Nombre, Especialidad)

VALUES

('Roberto Díaz', 'Reparación de Hardware'),

('Sofía Herrera', 'Mantenimiento de Software'),

('Manuel Acosta', 'Redes y Conectividad'),

('Patricia Soto', 'Periféricos y Accesorios'),

('David Rojas', 'Diagnóstico General')

GO

-- Insertar en Reparaciones

INSERT INTO Reparaciones (EquipoID, FechaSolicitud, Estado)

VALUES

(1, '05/07/2026', 'Pendiente'),

(2, '06/07/2026', 'Pendiente'),

(3, '07/07/2026', 'En proceso'),

(4, '08/07/2026', 'Pendiente'),

(5, '09/07/2026', 'En proceso')

GO

-- Insertar en Asignaciones

INSERT INTO Asignaciones (ReparacionID, TecnicoID, FechaAsignacion)

VALUES

(1, 1, '05/07/2026'),

(2, 2, '06/07/2026'),

(3, 3, '07/07/2026'),

(4, 4, '08/07/2026'),

(5, 5, '09/07/2026')

GO

-- Insertar en DetallesReparacion

```
INSERT INTO DetallesReparacion (ReparacionID, Descripcion, FechaInicio, FechaFin)
```

```
VALUES
```

```
(1, 'Equipo no enciende, revisión de fuente de poder', '05/07/2026', NULL),
```

```
(2, 'Atasco de papel y limpieza de cabezales', '06/07/2026', NULL),
```

```
(3, 'Problema con batería y bajo rendimiento', '07/07/2026', NULL),
```

```
(4, 'Pantalla táctil no responde correctamente', '08/07/2026', NULL),
```

```
(5, 'Sistema operativo lento y con errores', '09/07/2026', NULL)
```

```
GO
```

```
-- =====
```

-- 2. OPERACIONES CRUD

```
-- =====
```

-- CREAR: Insertar un nuevo registro

```
INSERT INTO Usuarios (Nombre, CorreoElectronico, Telefono)
```

```
VALUES ('Pedro López', 'pedro.lopez@correo.com', '8234-5678') -- Agrega un nuevo usuario
```

```
GO
```

-- LEER: Consultar registros

```
SELECT * FROM Usuarios -- Muestra todos los usuarios registrados
```

```
GO
```

-- ACTUALIZAR: Modificar un dato existente

```
UPDATE Usuarios SET Telefono = '8234-9999' WHERE UsuarioID = 6 -- Cambia el teléfono del  
usuario 6
```

```
GO
```

-- BORRAR: Eliminar un registro

```
DELETE FROM Usuarios WHERE UsuarioID = 6 -- Elimina el usuario número 6
```

```
GO
```

```
-----
```

Todos los comandos CRUD fueron incluidos en el archivo de SQL al final de este.

En color amarillo, los puntos que no se desarrollaran según indicación del profesor.

Procedimientos Almacenados: Crear colecciones precompiladas de sentencias SQL para tareas repetitivas:

sp_RegistrarReparacion: Un procedimiento que reciba el EquipoID y la FechaSolicitud e inserte una nueva fila en la tabla Reparaciones. sp_AsignarTecnico: Que vincule una reparación con un técnico disponible en la tabla Asignaciones.

Análisis Lógico: Implementar un Trigger que, al finalizar una reparación (en DetallesReparacion), cambie automáticamente el Estado en la tabla Reparaciones de "Pendiente" a "Completado".

Fase 4: Consultas de Análisis e Informes

Para demostrar que la información está "en la punta de sus dedos", deben generar las siguientes consultas (queries) complejas:

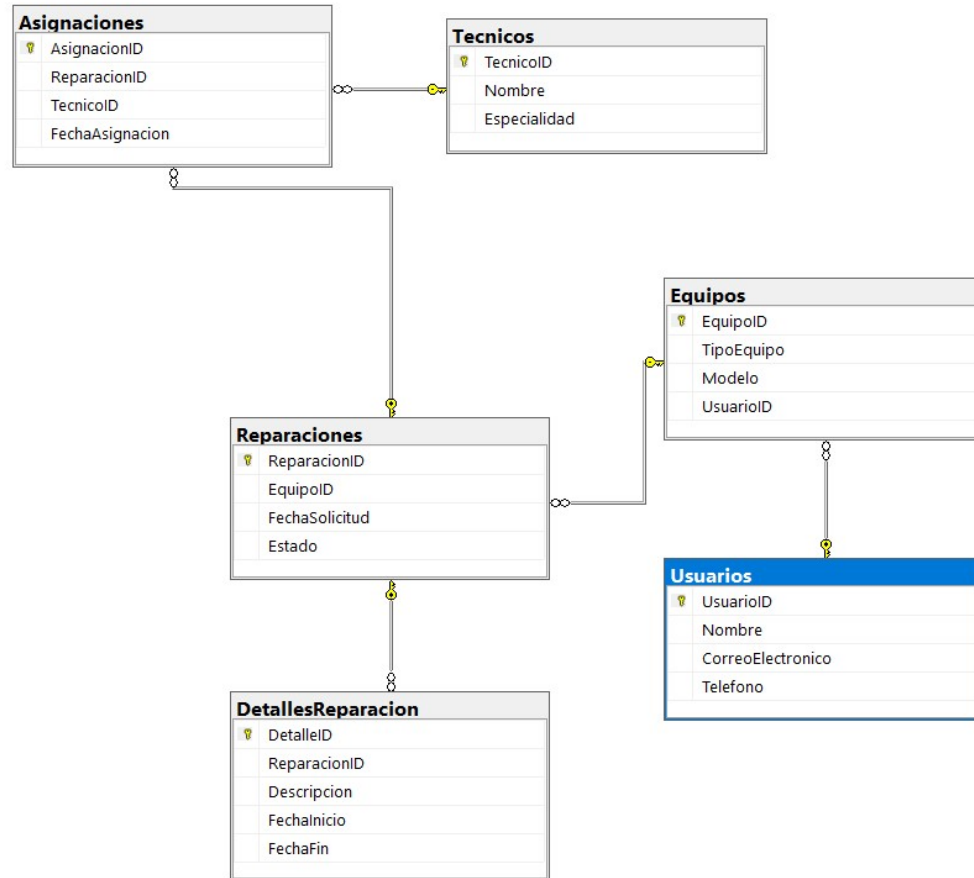
Informe de Asignaciones: Usar INNER JOIN para mostrar una lista con el nombre del usuario, el modelo de su equipo y el nombre del técnico que tiene asignada la reparación.

Estadística de Carga de Trabajo: Utilizar GROUP BY y la función COUNT() para mostrar cuántas reparaciones tiene asignadas cada técnico.

Filtro de Seguridad: Crear una consulta con la cláusula WHERE para localizar todos los equipos de un Modelo específico que ingresaron en el último mes.

DIAGRAMA DE COMO QUEDO LA BASE DE DATOS

DESKTOP-73... Diagram_0* X Proyecto Base...istian P (78))



DOCUMENTO DE ANALISIS

Justificación técnica del porque se eligieron ciertos tipos de datos y como se resolvieron los conflictos de llaves foráneas.

Se eligieron los tipos de datos buscando eficiencia, precisión, reducción de espacio en disco y cumplimiento de las reglas del negocio, tal como se detalla a continuación:

Tipo de dato	Campos donde se utiliza	Justificación técnica
INT	Todas las claves primarias y foráneas (ID)	Es un tipo numérico entero que ocupa solo 4 bytes, permite búsquedas rápidas, es compatible con todos los motores de bases de datos y soporta valores lo suficientemente altos para no agotarse en el uso normal del sistema.
IDENTITY(1,1)	Claves primarias de cada tabla	Genera valores únicos y secuenciales de forma automática, evita errores al ingresar identificadores manualmente y garantiza que cada registro tenga una referencia única.
VARCHAR(n)	Nombres, correos, modelos, descripciones, estados	Almacena texto de longitud variable: solo ocupa el espacio necesario según el contenido ingresado, a diferencia de CHAR que reserva espacio fijo. Se definió un tamaño máximo acorde al dato que se va a guardar para evitar desperdicio de memoria.
NOT NULL	Campos obligatorios (nombre, correo, modelo, etc.)	Impide que queden registros incompletos, manteniendo la integridad de la información y evitando errores en consultas o procesos posteriores.
UNIQUE	Campo CorreoElectronico	Garantiza que no existan dos usuarios con el mismo correo, evitando duplicidad de registros y asegurando que cada usuario se identifique de forma inequívoca.
DATETIME	Campos de fechas (FechaSolicitud, etc.)	Permite almacenar fecha y hora en un solo campo, es el estándar para este tipo de información y admite el formato de ingreso dd/mm/aaaa que se solicitó.
NULL	Campos opcionales (Telefono, FechaInicio, FechaFin)	Permite dejar el campo vacío cuando el dato aún no se tiene o no aplica, sin generar errores al guardar el registro.

Como se resolvieron los inconvenientes o conflictos con las llaves foráneas

Las llaves foráneas establecen vínculos entre tablas, asegurando que un dato relacionado solo pueda existir si ya está registrado en su tabla principal. Esto evita registros huérfanos (datos que hacen referencia a algo que ya no existe) y mantiene la coherencia de toda la información.

En todo momento se garantiza que no se puede ingresar un valor en una llave foránea que no exista en la tabla principal. Por ejemplo: no se puede registrar un equipo con un UsuarioID que no está creado. Esto evita inconsistencias desde el momento del ingreso de datos.

Por ejemplo:

Al intentar eliminar un registro:

Si se intenta borrar un registro que tiene referencias en otras tablas (por ejemplo: borrar un usuario que ya tiene equipos registrados), la base de datos bloquea la operación y muestra un mensaje de error.

Solución: Para poder eliminar ese registro, primero se deben borrar o modificar todos los registros que dependen de él. Esto evita perder información de forma accidental.

Al intentar modificar un identificador:

Si se intenta cambiar el valor de una clave primaria (por ejemplo: cambiar el UsuarioID), la base de datos bloquea la modificación si hay registros relacionados.

Solución: Los identificadores son valores únicos que no deben cambiarse una vez creados, por lo que esta restricción refuerza una buena práctica de diseño. Si fuera necesario modificarlo, se debe actualizar primero manualmente todos los registros relacionados.

También se realizaron pruebas con código CRUD con comandos para insertar, borrar, actualizar un registro y realizar consultas de los registros, lo que permitió observar en mayor detalle el funcionamiento de las llaves foráneas y sus resultados en la consulta realizada.

Requisitos de Entrega

Script SQL (.sql): Con comentarios que expliquen la lógica de cada procedimiento y trigger.

Documento de Análisis: Justificación técnica de por qué se eligieron ciertos tipos de datos y cómo se resolvieron los conflictos de llaves foráneas.

Subida a Repositorio: El código debe estar disponible en un repositorio de GitHub, documentando los comandos de git commit y git push utilizados para la colaboración grupal.